

Go2 Reinforcement Learning Locomotion and Sim-to-Real Transfer

Richard Albert King Mechoda

M.Sc. Robotics Engineering, DIBRIS, University of Genoa, Italy

Student ID: 8525970 | Cognitive Architectures for Robotics (COGAR)

Assignment 13 (E4): RL-Based Locomotion Training and Sim-to-Real Deployment (Simulation and Real Robot)

Abstract. We develop, train, validate and deploy a reinforcement-learning locomotion controller for the Unitree Go2 quadruped. The policy is trained *entirely in simulation* (NVIDIA Isaac Lab, PPO via RSL-RL, 16,384 parallel robots, $\approx 2 \times 10^9$ steps, 1 h 40 min on one RTX PRO 6000) and transferred *zero-shot* to the physical robot through the Unitree SDK and a ROS 2 Humble interface. The best flat-terrain policy reaches a 99.7% episode survival rate (0.33% falls), a final return of ≈ 31.2 , and a 0.22 m/s velocity-tracking error in simulation, and walks on the real Go2 EDU with no real-world fine-tuning, both from a tethered laptop and from the onboard Jetson Orin NX. A rough-terrain policy with a height-scan observation is trained over stairs and uneven blocks and validated in simulation; its hardware deployment is pending. We compare two policy configurations to isolate the decisive sim-to-real factor, an asymmetric actor-critic, and we analyse the sim-to-real gap, its failure cases and its limitations. Following the COGAR course, we read the system through the Sense-Reasoning/Plan-Act loop: the learned policy is a *reactive behaviour* (a sensory-to-motor mapping), the deployment state machine is a *fixed-priority behaviour coordinator* whose safety reflexes *suppress* the walking behaviour, and several modules instantiate the course’s software design patterns. A short demonstration video is available online.¹

Index Terms. quadruped locomotion, reinforcement learning, sim-to-real transfer, domain randomization, reactive architectures, behaviour-based robotics.

I. RESEARCH PROBLEM

Hand-designing a controller that makes a legged robot walk is difficult, and training one directly on the physical robot is worse: it is slow, costly and damaging, because learning requires thousands of falls. The now-standard alternative, used by the leading legged-robotics groups, is to *learn the controller in simulation*, where thousands of robots can be stepped in parallel on one GPU [1], and then transfer the result to the real robot. The central difficulty is the **sim-to-real gap**: a policy that is excellent in a physics simulator can fail on hardware, because the simulator is not the world. Friction, actuator latency, motor-gain errors, mass and contact dynamics

all differ, and some quantities the simulator provides for free, such as the body’s linear velocity, cannot be measured on the robot.

This project addresses that problem for the Unitree Go2 quadruped and, as the COGAR assignment requires, frames it within *Cognitive Architectures for Robotics*. The objectives, taken from the assignment brief, are to: set up the Go2 in Isaac Lab and configure the PPO/RSL-RL pipeline; train flat-terrain walking, trotting and velocity tracking; apply domain randomization (friction, mass, motor strength, terrain perturbations, observation noise, action delay); validate in Isaac Sim under perturbations; compare at least two policy configurations; evaluate velocity-tracking error, falls, orientation stability, cost of transport and disturbance robustness; export to ONNX/TorchScript and deploy on the real Go2 through the Unitree SDK and ROS 2 Humble; run real experiments and compare to simulation; and analyse the sim-to-real gap, its limitations and possible improvements. Section IV adds the cognitive-architecture reading the course asks for.

II. STATE OF THE ART

Methods that close the sim-to-real gap for learned legged locomotion can be organised by *where* they attack it (Table I). The *simulator-fidelity* family reduces the gap at its source: Tan et al. [3] transfer Minitaur gaits using accurate actuator and latency models, and Hwangbo et al. [4] learn the ANYmal actuator dynamics from real data. *Domain randomization*, introduced for perception by Tobin et al. [5] and extended to dynamics by Peng et al. [6], trains over a distribution of physics so that reality becomes one more sample. *Privileged learning* changes what the learner observes: the asymmetric actor-critic of Pinto et al. [7] gives the critic full simulator state while the actor sees only deployable observations, and teacher-student distillation [8], [9] compresses a privileged teacher into a proprioceptive student. Finally, massively parallel simulators [1] make wide randomization affordable. This project combines all four families; its distinctive choices, an asymmetric actor-critic and aggressive actuator-realism randomization, are detailed in Sections III and V.

III. RESEARCH FORMULATION

A. Hypotheses

We state three hypotheses before the experiments. **H1**: in simulation the gap is invisible, so well-tuned policies all look

¹Demonstration (Isaac Lab training to real deployment): <https://drive.google.com/file/d/1e1jpoKKcblSyVp17HyGn1CQgQ9hJk3pM/view?usp=sharing>

TABLE I: Four families of sim-to-real approaches for learned legged locomotion.

Family	Idea / representative works	Main limitation
Simulator fidelity / system ID	measured latency, learned actuator nets (Tan [3], Hwangbo [4])	per-robot effort; models age
Domain randomization	train over a physics distribution (Tobin [5], Peng [6])	range choice is ad hoc
Privileged learning / adaptation	privileged critic or teacher (Pinto [7], Lee [8], Miki [9])	added training complexity
Massive parallel simulation	cheap GPU experience (Rudin [1])	compute, not design

TABLE II: Actor observation (45 dims, asymmetric configuration).

Component	Dim	Source
Base angular velocity	3	IMU gyroscope
Projected gravity	3	IMU quaternion
Velocity command (v_x, v_y, ω_z)	3	operator / planner
Joint positions (rel. to stance)	12	encoders
Joint velocities	12	encoders
Last action	12	internal memory
Total	45	

excellent and a simulation-only evaluation is insufficient. **H2:** on hardware, a policy that depends on a quantity the robot cannot measure, the base linear velocity, drifts, because zeroing that input at deployment puts the policy out of distribution; an *asymmetric* design that confines the privileged input to the critic walks straight. **H3:** heavier action-smoothness penalties and actuator-latency randomization remove the motor chatter seen on hardware.

B. Learning formulation

We use Proximal Policy Optimization (PPO) [2], which maximises the expected discounted return $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_t \gamma^t r_t]$ through the clipped surrogate objective $L = \mathbb{E}_t [\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t)]$, with probability ratio ρ_t , $\epsilon = 0.2$, advantages from GAE ($\gamma = 0.99$, $\lambda = 0.95$), and a KL-adaptive learning rate (target KL 0.01). Actor and critic are separate MLPs $\rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow$ with ELU activations, about 100k parameters each, small enough for 50 Hz inference on the onboard computer.

C. Observation, action and control

The **actor** observes only what the real robot can measure (Table II): body angular velocity and projected gravity from the IMU, joint positions and velocities from the encoders, the velocity command, and the previous action. Projected gravity replaces raw orientation as a singularity-free, yaw-invariant “which way is down.” The **critic** additionally receives the privileged base linear velocity (Section IV). The policy outputs twelve joint-position offsets from a nominal stance, $q_{\text{target}} = q_{\text{default}} + 0.25 a$, tracked by a 500 Hz PD loop ($K_p=40$, $K_d=1.0$ at deployment). Inference runs at 50 Hz, physics at 200 Hz (decimation 4).

TABLE III: Reward terms (final flat configuration).

Positive	w	Negative	w
lin. vel. tracking	+1.5	flat orientation	-2.5
ang. vel. tracking	+0.75	vertical velocity	-2.0
foot air time	+0.5	action rate	-0.25
		undesired contacts	-1.0
		feet slide	-0.25
		joint deviation	-0.1
		torques / accel. / ang.xy	small

TABLE IV: Domain randomization ranges (per episode).

Parameter	Range
Body mass	-2 to +4 kg
Centre of mass	± 5 cm (xy), ± 2 cm (z)
Friction (static / dynamic)	0.4-1.4 / 0.3-1.0
Actuator gains K_p, K_d	$\pm 20\%$
Actuator latency	0-20 ms (DelayedPDActuator)
Joint friction / armature	randomised
External force / push	± 3 N / ± 0.5 m/s every 10-15 s
Stand-still training	20% of robots, zero command

D. Reward and emergent gait

The reward is a weighted sum of twelve terms (Table III). No footstep pattern is prescribed; the trot gait *emerges* from optimising “track the command, stay upright, lift the feet, do not waste energy.”

E. Domain randomization

Every episode perturbs the physics (Table IV): mass, centre of mass, friction, restitution, external forces and pushes, *actuator gains* $\pm 20\%$, joint friction and armature, and *actuator latency* 0-20 ms via Isaac Lab’s DelayedPDActuator. Twenty percent of robots receive a zero command (stand-still training), so the policy also learns to hold station. The real robot becomes one variant in this distribution.

F. Two configurations and the flat/rough policies

To satisfy the comparison requirement we train two configurations that differ only in the observation architecture and actuator-realism randomization. **Config A (V2)** is *symmetric*: a 48-dim actor that includes the base linear velocity. **Config B (V3)** is *asymmetric*: a 45-dim actor (base velocity given only to the critic), plus the delayed actuator, $\pm 20\%$ gain randomization, joint-friction randomization, stand-still training, and a $5\times$ heavier action-rate penalty. Network, PPO hyper-parameters, parallel environments and budget are identical. Separately, two terrains are trained: a **flat** policy (45-dim) and a **rough** policy that adds a 187-dim height scan and is trained over stairs and uneven blocks in all directions (Fig. 1, right).

G. Deployment

The trained policy is exported to ONNX (452 KB) and TorchScript. A verified permutation (SIM_TO_REAL = [3, 0, 9, 6, 4, 1, 10, 7, 5, 2, 11, 8]) reconciles the Isaac Lab joint order (PhysX BFS) with the Unitree per-leg order. On the robot, a 50 Hz loop reads the IMU and encoders, builds the 45-dim observation (the base linear velocity is not needed; it

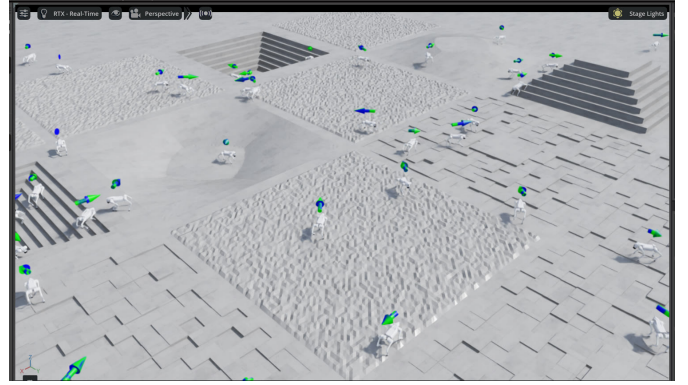
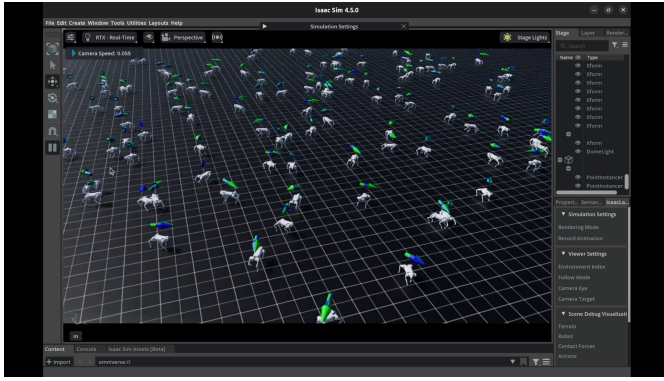


Fig. 1: Training in Isaac Lab with 16,384 parallel Go2 robots (arrows show each robot’s commanded velocity). Left: flat terrain, deployed zero-shot on the real robot. Right: rough terrain with stairs and uneven blocks, validated in simulation; hardware deployment is pending.

is critic-only), runs ONNX inference (~ 1 ms), and sends joint targets to the 500 Hz driver PD loop; the built-in sport mode is released first so it does not override low-level control. The same policy was run both from a laptop over Ethernet and from the onboard Jetson Orin NX, with consistent behaviour; the Jetson allows fully untethered operation.

IV. COGNITIVE APPROACH

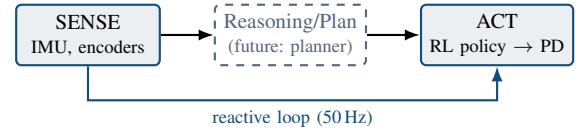
The COGAR course frames every robot architecture on the same **Sense** $\rightarrow$ **Reasoning/Plan** $\rightarrow$ **Act** loop and distinguishes four families: *hierarchical* (Sense-Plan-Act, all three stages in sequence), *reactive* (the Reasoning/Plan stage removed), *behaviour-based* (parallel sense-act behaviours with an arbitration step), and *hybrid reactive-deliberative*, also called cognitive (a direct Sense $\leftrightarrow$ Act loop with Reasoning/Plan on top). This section places the project on that map using the course’s own vocabulary, and connects the implementation to the course’s design patterns and to Marr’s three levels of analysis.

A. The project is a reactive architecture

At run time the locomotion policy is a tight **Sense** $\rightarrow$ **Act** loop with the *Reasoning/Plan stage removed* (Fig. 2): it senses (IMU and encoders), maps the observation directly to motor targets, and never plans or maintains a world model. In the course taxonomy this is exactly the *reactive* family. The course defines a **behaviour** as “a mapping of sensory inputs to a pattern of motor actions used to achieve a task,” decomposed into a *perceptual schema* and a *motor schema*; the policy is precisely such a behaviour, with the observation construction as its perceptual schema and the joint-offset output (through the PD loop) as its motor schema. The course’s System 1/System 2 reading also fits: the policy is a *Type-1, reactive* behaviour that has been “compiled down” from a slow optimisation, which is what reinforcement learning literally does, turning deliberative credit assignment at training time into a fast model-free reaction at run time.

B. Behaviour coordination and subsumption

A single reactive behaviour is not enough on hardware: something must decide *when* it runs and stop it before it



the Reasoning/Plan stage is bypassed at run time (velocity command stands in for it; a planner plugs in here)

Fig. 2: The deployed controller on the course’s Sense-Reasoning/Plan-Act loop. The Reasoning/Plan stage is removed (dashed): the system is a *reactive* architecture, with the velocity command as the interface where a deliberative layer would later attach.

breaks the robot. The deployment finite-state machine IDLE $\rightarrow$ STAND_UP $\rightarrow$ READY $\rightarrow$ WALK $\rightarrow$ DAMPING $\rightarrow$ STOPPED provides this, and it is a textbook instance of the course’s **behaviour coordination**. It is *competitive*, winner-take-all coordination by *fixed priority*: a small set of safety reflexes (a tilt reflex above 46° , a 100 ms sensor watchdog, a NaN/Inf guard, action and joint clipping) sit above the walking behaviour and, when triggered, *suppress* it, in the precise sense of Brooks’ subsumption taught in the course, where suppression replaces the signal a behaviour sends to the actuators. These reflexes are also “reflexes” in the ethological sense the course gives: rapid, automatic, involuntary responses correlated with stimulus strength, the kind the course notes are “used for locomotion.” The system therefore realises the lower part of a behaviour-based architecture: a learned reactive skill, coordinated and protected by a fixed-priority arbiter.

C. Realising the course’s design patterns

The deployment code is built from the software design patterns the course teaches for robot architectures. The sensing path is the **Sensor-device** pattern (the IMU and encoder readers are pure, periodic data producers wrapping the SDK driver) feeding a **Publish-subscribe** channel (the Unitree DDS / ROS 2 topics push state to the controller, which never assumes who is listening). The policy step is the **Computational** pattern: one periodic `on_update()` that collects all input

TABLE V: Simulation metrics (best flat policy).

Metric	Value
Episode survival rate	99.7%
Fall rate (base contact)	0.33%
Velocity tracking error	0.22 m/s
Angular velocity tracking error	0.15 rad/s
Final mean return	≈ 31.2

ports (the observation), runs a single inference, and publishes all outputs (the joint targets), with no blocking calls in the loop. Most directly, the joint-order remapping is the **Adapter** pattern in its textbook form, translating between two index conventions (Isaac Lab BFS and the Unitree per-leg order) exactly as the course’s Adapter example translates between coordinate systems.

D. Marr’s levels and the reference architecture

The course uses Marr’s three levels as the bridge from biology to software. Read that way, the project is: *Level 1* (*what*), a quadruped should walk and recover, an existence proof from ethology; *Level 2* (*how, as processes*), the perceptual and motor schemas of the policy behaviour plus the fixed-priority coordinator (a finite-state machine); and *Level 3* (*implementation*), ONNX inference on the Jetson with a PD loop on the motor driver. Within the course’s reference cognitive architecture (Darvish et al. [11]), which maps Task Representation/Manager and the Knowledge Base to Reasoning/Plan and the Execution Manager, Path Planner and Controller to Act, this project implements the **Act / Controller** role: the reusable locomotion competence on which a future Reasoning/Plan layer (a navigation or task planner issuing velocity commands, the hybrid reactive-deliberative family) would stand. Task and motion planning, which the course raises in the hierarchical lecture, is the natural way to close that loop.

V. RESULTS AND CONGRUENCE

A. Simulation validation and the two-config comparison

Both configurations were replayed in Isaac Sim across command directions, the stand-still command and perturbations (pushes, friction changes, added payload). The best flat policy reaches the metrics in Table V: a 99.7% survival rate, a 0.22 m/s velocity-tracking error, and a final return of about 31.2. In simulation the two configurations are nearly indistinguishable, which confirms **H1**: the gap is invisible in simulation, so a simulation-only study would not reveal it.

B. Real-robot deployment

On the physical Go2 EDU the same network that finished training runs unchanged at 50 Hz (Fig. 3). Config A drifts sideways; Config B walks straight, confirming **H2**: the asymmetric actor-critic, by keeping the policy inside its own perceptual horizon, removes the drift. The heavier action-rate penalty and the latency randomization remove the motor chatter, confirming **H3**. With a neutral command the policy holds station rather than trotting in place, the effect of stand-still training. The rough-terrain policy is validated in simulation,



Fig. 3: Zero-shot deployment. Left: Isaac Sim validation. Right: the real Unitree Go2 EDU walking under joystick command, running the same ONNX policy that finished training.

TABLE VI: Sim-to-real failures and fixes.

Failure	Root cause	Fix
Foot dragging	air-time reward too low	$\times 40$ air-time, feet-slide penalty
Collapsing	orientation penalty too harsh	rebalance: tracking dominates
Motor chatter	actions change too fast	$\times 5$ action-rate, accel. penalty
Sideways drift	actor used base velocity it cannot measure	asymmetric actor-critic
Gain sensitivity	real latency, gains, stiction	DelayedPDActuator, $\pm 20\%$ gains
Twisted motion	sim vs SDK joint order	verified joint remap (Adapter)

including stair climbing, but is not yet deployed on hardware, because the Go2 used has no depth sensor feeding the height-scan observation.

C. Sim-to-real gap analysis

The final policy is the result of fixing a sequence of failures, each exposing one mismatch (Table VI); the version history $V1 \rightarrow V2 \rightarrow V3$ records the process. The largest single improvement was the asymmetric actor-critic (failure four).

D. Congruence of results and conclusions

We keep the claims conservative. *Zero-shot flat walking on the real Go2* is supported by the demonstration and by the disappearance of the specific failure modes. *The asymmetric actor-critic removes drift* is supported by the V2-versus-V3 comparison (H2). *Domain randomization bridges the gap* is supported by robustness across the training distribution. The honest scope is also stated: the simulation metrics are quantitative, whereas the hardware comparison is qualitative, and a logged real-robot benchmark (cost of transport, measured tracking, push recovery) remains future work; the rough-terrain policy is validated only in simulation; and, in cognitive-architecture terms, the project realises the reactive behaviour and its fixed-priority coordinator, not a deliberative Reasoning/Plan layer. Adding that layer, a planner issuing the velocity commands the policy already consumes, would complete the hybrid reactive-deliberative architecture and is

the clear next step, together with a depth sensor for on-hardware rough-terrain locomotion and a learned estimator for the base velocity.

ACKNOWLEDGMENTS

The author thanks Prof. Fulvio Mastrogiovanni and Gianluca Galvagni for the Cognitive Architectures for Robotics course and for their guidance and support throughout this project.

REFERENCES

- [1] N. Rudin et al., “Learning to walk in minutes using massively parallel deep RL,” *CoRL*, 2022.
- [2] J. Schulman et al., “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.
- [3] J. Tan et al., “Sim-to-real: Learning agile locomotion for quadruped robots,” *RSS*, 2018.
- [4] J. Hwangbo et al., “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, 2019.
- [5] J. Tobin et al., “Domain randomization for transferring deep neural networks from simulation to the real world,” *IROS*, 2017.
- [6] X. B. Peng et al., “Sim-to-real transfer of robotic control with dynamics randomization,” *ICRA*, 2018.
- [7] L. Pinto et al., “Asymmetric actor critic for image-based robot learning,” *RSS*, 2018.
- [8] J. Lee et al., “Learning quadrupedal locomotion over challenging terrain,” *Science Robotics*, vol. 5, no. 47, 2020.
- [9] T. Miki et al., “Learning robust perceptive locomotion for quadrupedal robots in the wild,” *Science Robotics*, vol. 7, no. 62, 2022.
- [10] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE J. Robotics and Automation*, vol. 2, no. 1, 1986.
- [11] K. Darvish et al., “A hierarchical architecture for human-robot cooperation processes,” *IEEE Trans. Robotics*, vol. 37, no. 2, 2021.
- [12] D. Marr, *Vision*. W. H. Freeman, 1982.